IMAGE PROCESSING AND ANALYSIS

CS30015



ASSIGNMENT-I

SUBMITTED BY: PRIYANSH MISHRA

ROLL: 24155789

SEC: IPA-12

ORIGINAL IMAGE USED



TASK-1

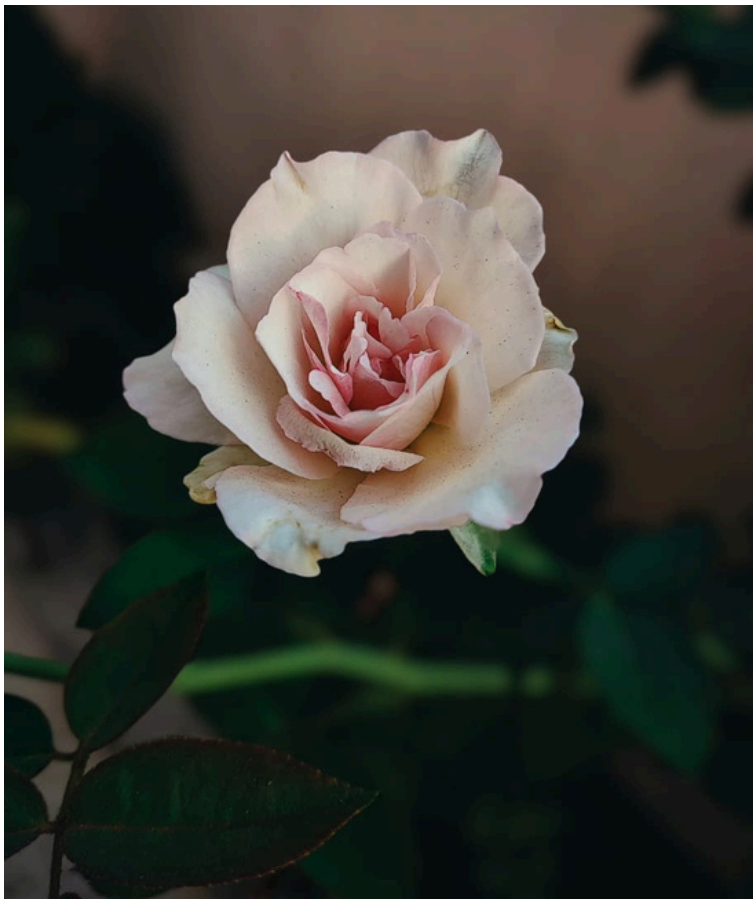
Display your photograph and use your code to find its width, height, number of color channels, and file size in KB or MB. Write the dimensions as width × height.

What it does?

This task loads the original photograph and examines its basic properties, including width, height, number of colour channels, and file size. These properties establish a baseline for comparing the image before and after applying different image processing techniques.

Observations

The original rose photograph has a resolution of 960 × 1280 pixels, with three colour channels and a file size of 45.11 KB. Its portrait orientation and colour information provide a baseline for evaluating changes in brightness, contrast, colour, and detail after applying different image processing techniques.



Input Image

```
--- ORIGINAL IMAGE ANALYSIS ---
Width: 960 pixels
Height: 1280 pixels
Dimensions: 960 x 1280
Channels: 3
File size: 45.11 KB
Saved: original.png
```

Output

CODE USED

```python
import cv2
import os

# Load the original image
image_path = r"C:\Users\priya\OneDrive\Desktop\IPA Assignment\images\original.jpeg"
output_dir = r"C:\Users\priya\OneDrive\Desktop\IPA Assignment\outputs"
image = cv2.imread(image_path)

if image is None:
    print("Error: Unable to load the image.")
else:
    # Get image properties
    height, width, channels = image.shape
    file_size = os.path.getsize(image_path) / 1024

    # Display image information
    print("--- ORIGINAL IMAGE ANALYSIS ---")
    print(f"Width: {width} pixels")
    print(f"Height: {height} pixels")
    print(f"Dimensions: {width} x {height}")
    print(f"Channels: {channels}")
    print(f"File size: {file_size:.2f} KB")
```

TASK-2

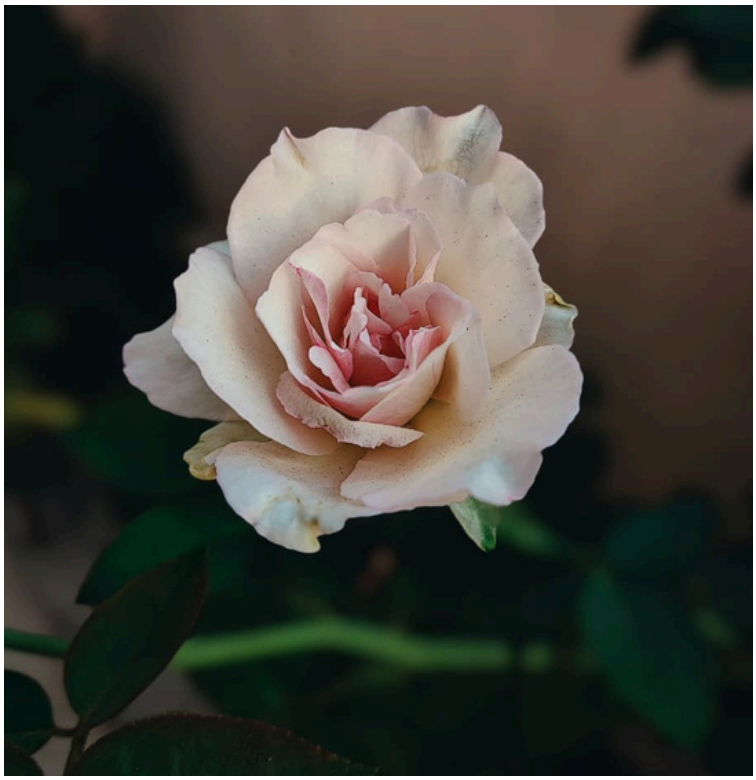
Convert your photograph to grayscale. Show the color and grayscale images side by side. Briefly explain what changes and what information is still visible.

What it does?

This task converts the original colour image into a grayscale image by removing colour information while preserving intensity variations. It uses OpenCV's cv2.cvtColor() function to convert the image from BGR to grayscale.

Observations

TThe colour image is converted into shades of gray, ranging from black to white, while preserving the intensity variations and major structural details of the rose.



Input Image



Output Image

CODE USED

```python
import cv2
import matplotlib.pyplot as plt

# Load the original image
image = cv2.imread("images/original.jpeg")

if image is None:
    print("Error: Unable to load the image.")
else:
    # Convert the image to grayscale
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Display original and grayscale images
    plt.figure(figsize=(10, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(gray_image, cmap="gray")
    plt.title("Grayscale Image")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

    # Save the grayscale image
    cv2.imwrite("outputs/grayscale.jpg", gray_image)

    print("Grayscale conversion completed successfully.")
```

TASK-3

Create a negative of the grayscale image. Then apply gamma correction with gamma values of 0.5, 1.0, and 2.0. For gamma correction, first scale the pixel values to the range [0, 1] and use s = r^gamma. Show the results and explain how each gamma value affects brightness.

Negative Transformation

What it does?

This task generates the negative of the original image by reversing the intensity of each pixel. Bright areas become dark, and dark areas become bright.

Observations

The transformation reverses the pixel intensities, producing an image with inverted colours and brightness while retaining the original dimensions.

Settings Used

- Intensity range: 0–255
- Transformation:

Negative = 255 − Original Pixel Value



Input Image



Output Image

CODE USED

```python
import cv2
import matplotlib.pyplot as plt

# Load the original image
image = cv2.imread("images/original.jpeg")

if image is None:
    print("Error: Unable to load the image.")
else:
    # Generate the negative image
    negative = 255 - image

    # Display original and negative images
    plt.figure(figsize=(10, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
    plt.title("Negative Image")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

    # Save the result
    cv2.imwrite("outputs/negative.jpg", negative)
```

Gamma Correction (0.5,1.0 and 2.0)

What it does?

Gamma correction adjusts an image's brightness using a nonlinear transformation. Different gamma values produce different brightness levels while retaining variations in image intensity.

Observations

For **γ = 0.5**: The image becomes brighter because lower intensity values are increased by the transformation.
For **γ = 1.0**: The normalized pixel intensities remain unchanged because raising a value to the power of one preserves it.
For **γ = 2.0**: The image becomes darker because normalized pixel values between zero and one decrease when squared.

Settings Used

Gamma value (γ)	Expected effect
0.5	Brightens the image
1	Preserves the original intensity values
2	Darkens the image



Input Image



Output Image
(0.5 Gamma Correction)



Output Image
(1.0 Gamma Correction)



Output Image
(2.0 Gamma Correction)

CODE USED

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the original image
image = cv2.imread("images/original.jpeg")
if image is None:
    print("Error: Unable to load the image.")
else:
    # Gamma values
    gamma_values = [0.5, 1.0, 2.0]
    # Display the original and corrected images
    plt.figure(figsize=(12, 8))
    plt.subplot(2, 2, 1)
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.title("Original Image")
    plt.axis("off")
    for i, gamma in enumerate(gamma_values, start=2):
        # Apply gamma correction
        normalized = image / 255.0
        corrected = np.power(normalized, gamma) * 255
        corrected = np.clip(corrected, 0, 255).astype(np.uint8)
        plt.subplot(2, 2, i)
        plt.imshow(cv2.cvtColor(corrected, cv2.COLOR_BGR2RGB))
        plt.title(f"Gamma = {gamma}")
        plt.axis("off")
        # Save the result
        cv2.imwrite(f"outputs/gamma_{gamma}.jpg", corrected)
    plt.tight_layout()
    plt.show()
    print("Gamma correction completed successfully.")
```

TASK-4

Apply linear contrast stretching and global histogram equalization separately to the grayscale image. Show each result beside the input. Plot the histogram of the input and of each result using the same intensity range. Are the darker and brighter details easier to see? Explain your observations.

Contrast Stretching

What it does?

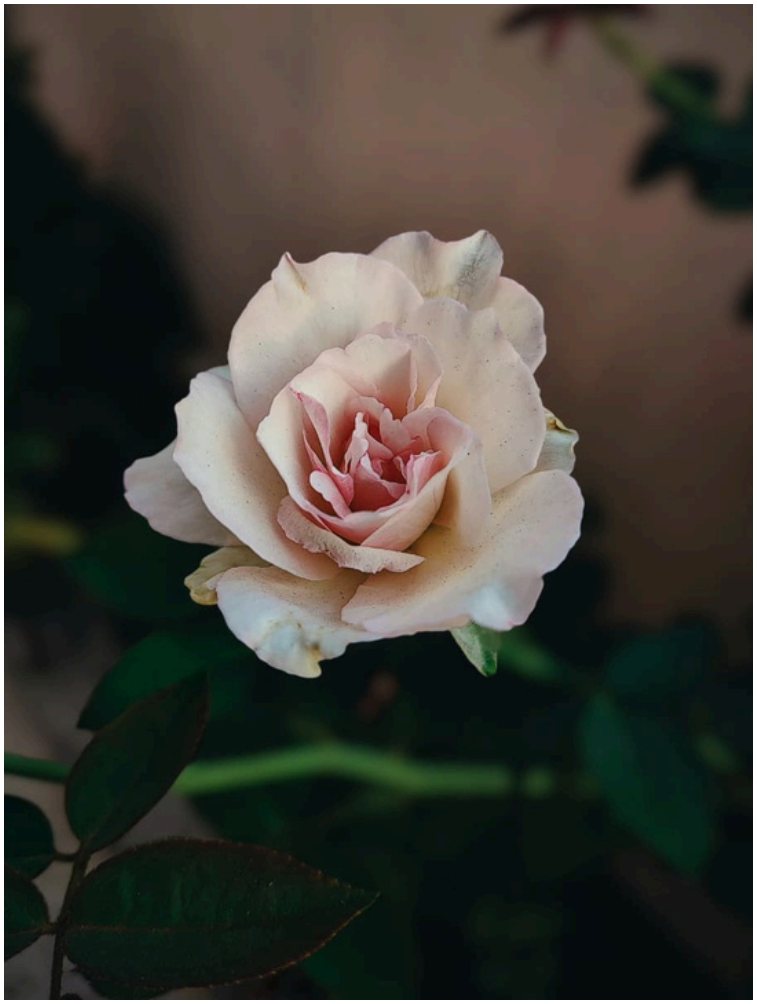
Contrast stretching improves an image's contrast by expanding its intensity values across a wider range. This helps distinguish details in darker and brighter regions.

Settings Used

- Input: Original colour image
- Intensity range: 0–255
- Method: Min–max normalization

Observations

Min–max normalization expands the pixel intensity range to 0–255, potentially improving the visibility of details and differences between darker and brighter regions.



Input Image



Output Image

CODE USED

```python
import cv2
import matplotlib.pyplot as plt

# Load the original image
image = cv2.imread("images/original.jpeg")

if image is None:
    print("Error: Unable to load the image.")
else:
    # Apply contrast stretching
    stretched = cv2.normalize(
        image, None, 0, 255, cv2.NORM_MINMAX
    )

    # Display the results
    plt.figure(figsize=(10, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(cv2.cvtColor(stretched, cv2.COLOR_BGR2RGB))
    plt.title("Contrast Stretched Image")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

    # Save the result
    cv2.imwrite("outputs/contrast_stretched.jpg", stretched)
```

Global Histogram Equalisation

What it does?

Histogram equalization improves image contrast by redistributing grayscale intensity values using the image's cumulative histogram. It can make details more visible in images with limited contrast.

Settings Used

- Input: Grayscale image
- Method: Histogram equalization using OpenCV's cv2.equalizeHist()
- Intensity range: 0–255

Observations

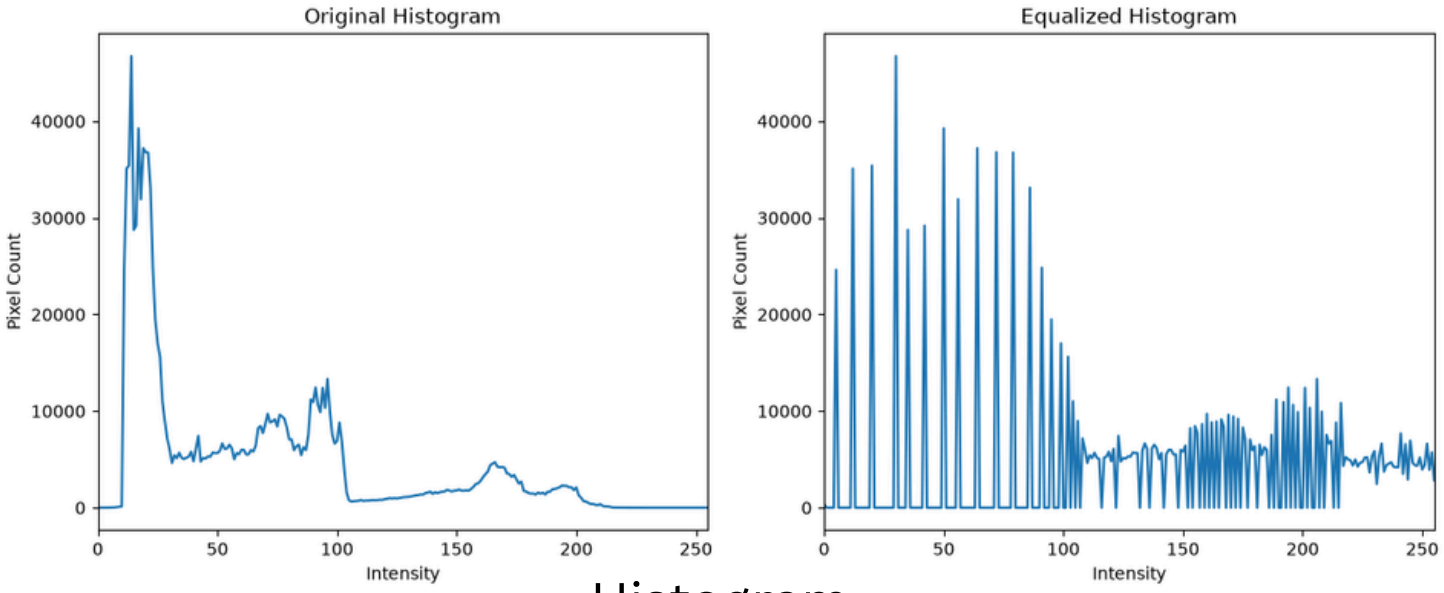
Histogram equalization redistributes grayscale intensity values to improve contrast. Depending on the original image's intensity distribution, it may reveal additional details or make some regions appear more pronounced.



Input Image



Output Image



Histogram

CODE USED

```python
import cv2
import matplotlib.pyplot as plt

# Load the original image
image = cv2.imread("images/original.jpeg")

if image is None:
    print("Error: Unable to load the image.")
else:
    # Convert the image to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Apply histogram equalization
    equalized = cv2.equalizeHist(gray)

    # Display the results
    plt.figure(figsize=(10, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(gray, cmap="gray")
    plt.title("Grayscale Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(equalized, cmap="gray")
    plt.title("Histogram Equalized Image")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

    # Save the result
    cv2.imwrite("outputs/histogram_equalized.jpg", equalized)
```

TASK-5

Smooth the grayscale image using Gaussian filters with 3 × 3 and 7 × 7 kernels. Mention the standard deviation used for each filter. Compare the results with the input. Describe how much blur you notice and whether any fine details are lost.

What it does?

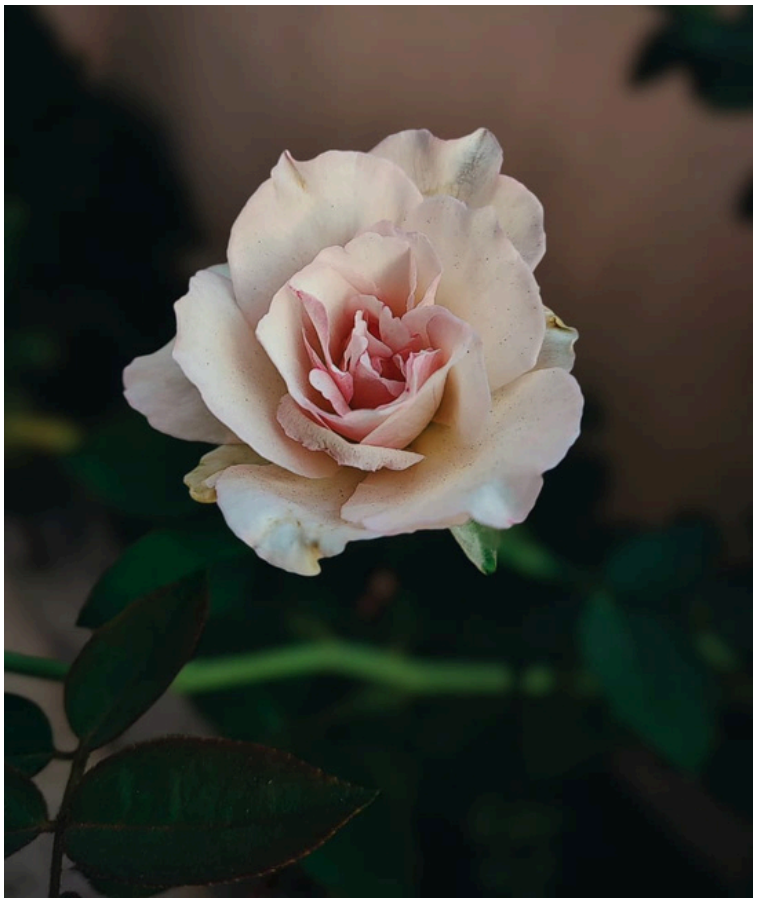
Gaussian smoothing reduces image noise and fine details by applying a Gaussian blur. It uses a kernel to calculate weighted averages of neighbouring pixels, producing a smoother image.

Settings Used

Setting	Configuration 1	Configuration 2
Kernel size	3 × 3	7 × 7
Standard deviation (σ)	0.8	1.5
Function	cv2.GaussianBlur()	cv2.GaussianBlur()

Observations

Configuration 1 — Kernel 3 × 3, σ = 0.8: Mildly smooths the image while retaining most of its fine details and edges.
Configuration 2 — Kernel 7 × 7, σ = 1.5: Produces stronger smoothing, suppressing more fine details and making edges appear less sharp.



Input Image



Output
(3x3 Gaussian Filter)



Output
(7x7 Gaussian Filter)

CODE USED

```python
import cv2
import matplotlib.pyplot as plt
# Load the original image
image = cv2.imread(r"C:\Users\priya\OneDrive\Desktop\IPA Assignment\images\original.jpeg")
if image is None:
    print("Error: Unable to load the image.")
else:
    # Apply Gaussian smoothing with different settings
    smooth_3 = cv2.GaussianBlur(image, (3, 3), 0.8)
    smooth_7 = cv2.GaussianBlur(image, (7, 7), 1.5)
    # Convert images from BGR to RGB
    original_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    smooth_3_rgb = cv2.cvtColor(smooth_3, cv2.COLOR_BGR2RGB)
    smooth_7_rgb = cv2.cvtColor(smooth_7, cv2.COLOR_BGR2RGB)
    # Display results
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 3, 1)
    plt.imshow(original_rgb)
    plt.title("Original Image")
    plt.axis("off")
    plt.subplot(1, 3, 2)
    plt.imshow(smooth_3_rgb)
    plt.title("Kernel 3x3, Sigma 0.8")
    plt.axis("off")
    plt.subplot(1, 3, 3)
    plt.imshow(smooth_7_rgb)
    plt.title("Kernel 7x7, Sigma 1.5")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
    # Save processed images
    cv2.imwrite("outputs/gaussian_3x3.jpg", smooth_3)
    cv2.imwrite("outputs/gaussian_7x7.jpg", smooth_7)
    print("Gaussian smoothing completed successfully.")
```

TASK-6

Use the Sobel operators to find intensity changes along the horizontal and vertical directions. Show both responses and the combined gradient magnitude. Describe the object boundaries you can see. If you rescale the responses for display, mention how you did it.

What it does?

Sobel edge detection identifies edges in an image by measuring changes in pixel intensity. It calculates gradients in the horizontal and vertical directions and combines them to determine the overall edge strength.

Settings Used

Parameter	Value
Input image	Grayscale image
Derivative order in X	1
Derivative order in Y	1
Kernel size	3 × 3
Output depth	64-bit floating point (CV_64F)

Observations

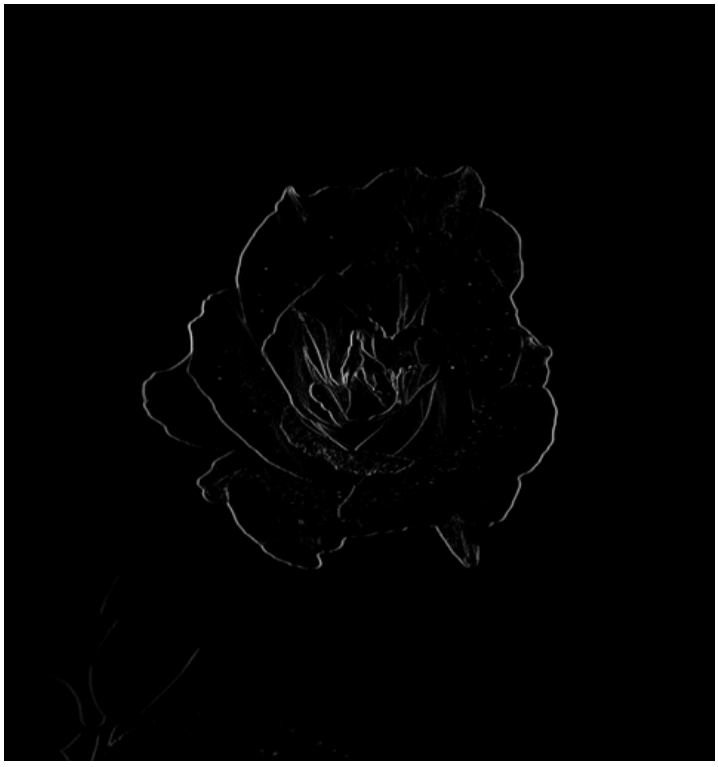
Sobel X — Vertical Edges: Highlights vertical boundaries by detecting intensity changes along the horizontal direction.
Sobel Y — Horizontal Edges: Highlights horizontal boundaries by detecting intensity changes along the vertical direction.
Gradient Magnitude: Combines the horizontal and vertical gradients to highlight edges in both directions. Stronger intensity indicates a stronger image gradient.



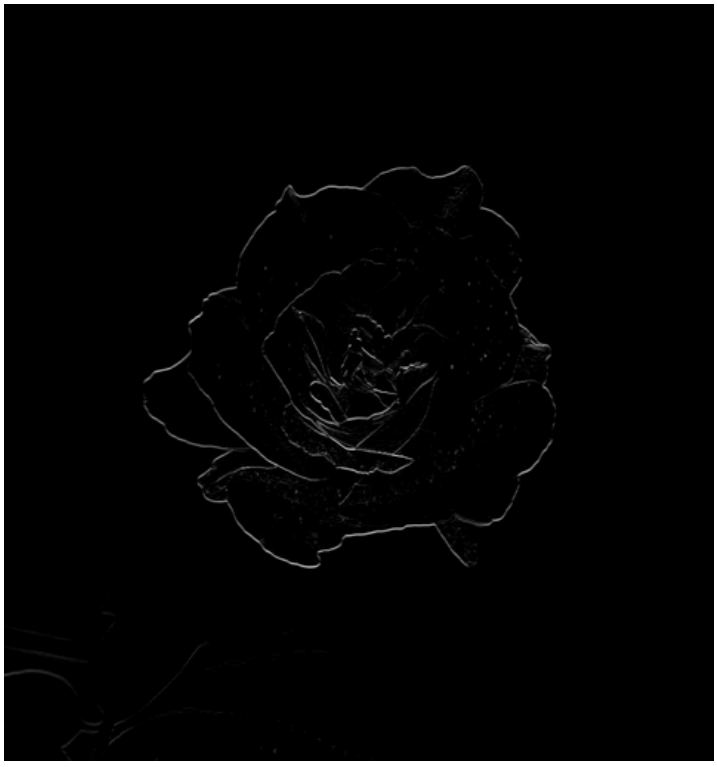
Input Image



Output Image
(Gradient Magnitude)



Output Image
(Sobel-X)



Output Image
(Sobel-Y)

CODE USED

```python
# Load the original image
import cv2
from matplotlib import pyplot as plt
import numpy as np
image = cv2.imread(r"C:\Users\priya\OneDrive\Desktop\IPA Assignment\images\original.jpeg")
if image is None:
    print("Error: Unable to load the image.")
else:
    # Convert to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # Apply Sobel operators
    sobel_x = cv2.Sobel(
        gray, cv2.CV_64F, 1, 0, ksize=3)
    sobel_y = cv2.Sobel(
        gray, cv2.CV_64F, 0, 1, ksize=3)
    # Calculate gradient magnitude
    magnitude = np.sqrt(sobel_x**2 + sobel_y**2)
    # Convert results for display
    sobel_x_display = cv2.convertScaleAbs(sobel_x)
    sobel_y_display = cv2.convertScaleAbs(sobel_y)
    magnitude_display = cv2.convertScaleAbs(magnitude)
    # Display results
    plt.figure(figsize=(12, 8))
    plt.subplot(2, 2, 1)
    plt.imshow(gray, cmap="gray")
    plt.title("Grayscale Image")
    plt.axis("off")
    plt.subplot(2, 2, 2)
    plt.imshow(sobel_x_display, cmap="gray")
    plt.title("Sobel X - Vertical Edges")
    plt.axis("off")
    plt.subplot(2, 2, 3)
    plt.imshow(sobel_y_display, cmap="gray")
    plt.title("Sobel Y - Horizontal Edges")
    plt.axis("off")
    plt.subplot(2, 2, 4)
    plt.imshow(magnitude_display, cmap="gray")
    plt.title("Gradient Magnitude")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
    # Save the output images
    cv2.imwrite(r"outputs\sobel_x.jpg", sobel_x_display)
    cv2.imwrite(r"outputs\sobel_y.jpg", sobel_y_display)
    cv2.imwrite(r"outputs\sobel_magnitude.jpg", magnitude_display)
    print("Sobel edge detection completed successfully.")
```

A SHORT REFLECTION

1. Which technique was most useful for your photograph, and why?
I found contrast stretching useful because it improves the difference between darker and brighter areas of the photograph. This can make the details of the rose more noticeable and help different regions stand out.

2. How did gamma correction differ from contrast stretching and histogram equalization?
Gamma correction changes the brightness of an image depending on the gamma value used. Contrast stretching expands the intensity range, while histogram equalization redistributes grayscale intensity values to improve contrast. Although all three affect the appearance of an image, they work in different ways.

3. How did the Gaussian kernel size affect fine details and edges?
The 3 × 3 kernel with a standard deviation of 0.8 applied mild smoothing and preserved most of the fine details. In comparison, the 7 × 7 kernel with a standard deviation of 1.5 produced a stronger blur, making small details less visible and edges softer.

4. Which techniques made details easier to see, and which helped you view the image in a different way?
Contrast stretching and histogram equalization can make details easier to distinguish by improving contrast. Gamma correction helps adjust brightness, while Sobel edge detection highlights the boundaries and structure of the rose. Image negative gives the photograph a different appearance by reversing its intensity values, whereas Gaussian smoothing reduces fine details and makes the image look smoother.

SOFTWARE AND LIBRARIES USED

This assignment was developed using Python in Visual Studio Code (VS Code). Python was used to write the program and perform different image processing operations. The following libraries helped implement and display the results:

- **OpenCV (cv2):** Used to load the original photograph and perform operations such as grayscale conversion, image negative, contrast stretching, histogram equalization, Gaussian smoothing, and Sobel edge detection.
- **NumPy:** Used for mathematical calculations and pixel intensity manipulation, particularly in gamma correction and gradient magnitude calculations.
- **Matplotlib:** Used to display the original and processed images, making it easier to compare the results of different techniques.

VS Code provided a convenient environment for writing and running the code. Together, these tools made it possible to process the photograph, observe the changes produced by each technique, and understand how image processing affects brightness, contrast, fine details, and edges.